Git and GitHub

1. Git Fundamentals

1.1 History of Git

- Git was created by **Linus Torvalds** in **2005**, the same person who created the Linux kernel.
- Before Git, the Linux kernel development community used a proprietary version control system called **BitKeeper**.
- BitKeeper was provided free of cost to the Linux community by its company, but in 2005 the relationship broke down after a dispute over licensing terms (a developer reverse-engineered BitKeeper's protocols, and the company withdrew free access).
- This forced Linus Torvalds to build a new version control system from scratch, one that would meet the specific needs of a massive, distributed, open-source project like Linux.
- Torvalds set clear goals for this new system:
  - **Speed** – operations should be fast even on huge codebases.
  - **Simple design**.
  - **Strong support for non-linear development** – meaning thousands of parallel branches.
  - **Fully distributed** – no central server required.
  - **Able to handle large projects efficiently** (like the Linux kernel, with thousands of contributors).
- The initial version of Git was developed in about **10 days**.
- The name "Git" is British slang for an unpleasant or foolish person. Torvalds jokingly said, "I'm an egotistical bastard, and I name all my projects after myself. First Linux, now Git."
- Since 2005, Git has grown into the most widely used version control system in the world, maintained by a large community of contributors, with Junio Hamano serving as the current lead maintainer.
- Git is **open-source** and free to use.

1.2 Git and GitHub

- **Git** is a version control **software/tool** installed and run locally on a computer.
- **GitHub** is a **cloud-based hosting service** for Git repositories. It provides a web interface, along with collaboration features (pull requests, issues, project boards, actions, etc.).
- Git works even without internet access (local operations); GitHub requires internet access.
- GitHub was founded in 2008 (built on top of Git) and was acquired by Microsoft in 2018.
- Other similar platforms exist: GitLab, Bitbucket — but GitHub is the most popular.

- Key distinction to remember: **Git = the tool (version control system)**; **GitHub = a company/platform that hosts Git repositories online**.

1.3 Introduction to Git

- Git is a **free, open-source Distributed Version Control System (DVCS)**.
- It is designed to track changes in source code (or any set of files) during software development.
- It allows multiple developers to work on the same project simultaneously without overwriting each other's work.
- Every developer has a complete copy of the project history on their local machine (called a "local repository").
- Git tracks changes using **snapshots**, not just differences (unlike many older systems that store only file differences/deltas).

1.4 Why Git?

- **Collaboration**: Enables multiple people to work on the same codebase at the same time.
- **Tracking history**: Keeps a complete history of every change made, who made it, and when.
- **Reverting changes**: Allows going back to any previous version of the project if something breaks.
- **Branching and merging**: Makes it easy to create separate lines of development (branches) for new features or experiments without affecting the main code, and later merge them back.
- **Backup**: Since every developer has a full copy of the repository, the project is naturally backed up in multiple places.
- **Industry standard**: Almost all modern software companies and open-source projects use Git, making it an essential skill.
- **Speed and efficiency**: Most operations are performed locally, making Git extremely fast compared to centralized systems that need constant network access.

1.5 Features of Git

- **Distributed architecture**: Every user has a full copy of the repository (including complete history), not just the latest snapshot.
- **Speed**: Local operations (commit, diff, log, branch) are very fast since no network call is required.
- **Branching and merging**: Lightweight, cheap, and fast branch creation; Git is known for its powerful branching model.
- **Data integrity**: Every file and commit is checksummed using **SHA-1 hashing** before being stored, ensuring data cannot be silently corrupted or lost.
- **Staging area (Index)**: A unique intermediate area where changes can be reviewed and organized before committing.

- **Non-linear development support**: Supports thousands of parallel branches.
- **Open-source and free**: Anyone can use, study, or modify Git.
- **Compatibility**: Works across all major operating systems (Windows, macOS, Linux).
- **Security**: Uses cryptographic hashing, making history tampering easily detectable.
- **Scalability**: Can handle very large projects with a huge number of files and contributors (e.g., Linux kernel).

2. Version Control Concepts

2.1 What is Version Control?

- **Version Control** (also called **Source Control**) is a system that records changes to a file or set of files over time, so specific versions can be recalled later.
- It allows tracking of:
  - **What** changes were made.
  - **Who** made the changes.
  - **When** the changes were made.
  - **Why** the changes were made (via commit messages).
- A **Version Control System (VCS)** is the software tool that manages this process (e.g., Git, SVN, Mercurial).

2.2 Need for Version Control

- **Multiple versions of files**: Without VCS, developers often end up with messy file names like `project_final.doc`, `project_final_v2.doc`, `project_final_ACTUAL.doc`, leading to confusion.
- **Collaboration problems**: Multiple people editing the same file simultaneously can overwrite each other's work.
- **No history tracking**: Without VCS, there's no record of who changed what and why.
- **Difficulty undoing mistakes**: Without version history, reverting to a previous working state is very hard or impossible.
- **Accountability**: Teams need to know which developer is responsible for a particular change or bug.
- **Parallel development**: Different features/bug fixes need to be developed at the same time without interfering with each other.

2.3 Benefits of Version Control

- **Complete history**: Every change is recorded, allowing you to see the entire evolution of a project.
- **Collaboration**: Multiple developers can work on the same project at the same time without conflicts.
- **Backup and recovery**: Since history is stored, files can never be truly "lost"; you can always revert to a previous state.
- **Branching and experimentation**: Developers can safely experiment with new ideas in isolated branches without risking the stable/main codebase.
- **Traceability and accountability**: Every change is linked to an author, timestamp, and message, making it easy to track responsibility.
- **Conflict resolution**: Helps identify and resolve conflicts when multiple people edit the same part of a file.
- **Improved productivity**: Reduces time wasted on manually managing file versions.
- **Supports auditing/compliance**: Useful in regulated industries where change history must be documented.

---

3. Types of Version Control Systems

3.1 Local Version Control System (LVCS)

- The simplest form of version control.
- Involves copying files into another directory (possibly with timestamps) to maintain different versions — this is essentially manual version control.
- Later, tools emerged that used a simple database on the local computer to keep track of all changes to files (e.g., **RCS – Revision Control System**).
- **Limitations**:
    - Works only on a single computer/local machine.
    - No collaboration possible — cannot be used by multiple people at different locations.
    - High risk of data loss if the local machine is damaged (no backup elsewhere).
    - Not scalable for team-based projects.

3.2 Centralized Version Control System (CVCS)

- Examples: **SVN (Subversion)**, **CVS (Concurrent Versions System)**, **Perforce**.
- Uses a **single central server** that stores all the versioned files and history.
- Developers ("clients") connect to this central server to get the latest version of files and to check in (commit) their changes.
- **Advantages over LVCS**:

    - Everyone knows to some extent what everyone else on the project is doing.
    - Administrators have fine-grained control over who can do what.
    - Easier to administer than having local databases on every client.
- **Disadvantages**:
    - **Single point of failure**: If the central server goes down, no one can collaborate, commit changes, or access version history at all.
    - If the server's disk becomes corrupted without proper backups, the entire project history can be lost.
    - Requires constant network/internet connectivity to perform most operations.
    - Slower operations since every action (commit, log, diff) may require communication with the server.

3.3 Distributed Version Control System (DVCS)

- Examples: **Git**, **Mercurial**, **Bazaar**.
- Clients do not just check out the latest snapshot of files — they **fully mirror the entire repository**, including its complete history.
- Every clone is essentially a **full backup** of all the project data.
- If any server dies, any of the client repositories can be copied back up to the server to restore it.
- Every checkout/clone is a complete backup of all the data.
- **Advantages**:
    - No single point of failure — since every collaborator has a full copy of the history.
    - Most operations (commit, log, diff, branch, merge) are performed **locally**, making them extremely fast and possible even without internet access.
    - Enables flexible workflows: developers can commit locally and sync with the remote server later.
    - Supports multiple remote repositories, enabling complex collaboration workflows (e.g., open-source projects with many contributors).

3.4 Centralized VCS vs Distributed VCS

Basis	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Repository	Single central repository	Every user has a full local copy of the repository
Examples	SVN, CVS, Perforce	Git, Mercurial, Bazaar
Point of failure	Single point of failure (central server)	No single point of failure

Basis	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Network dependency	Requires network connection for most operations (commit, log, history)	Most operations work offline (local commits, history, branching)
Speed	Slower (network-dependent operations)	Faster (local operations)
Backup	Only server has full history (needs separate backup strategy)	Every clone is a full backup automatically
Branching/Merging	Usually heavier and harder	Lightweight, fast, and encouraged
Collaboration model	Client-server model	Peer-to-peer model (any repo can sync with any other)
Access control	Easier centralized control by admin	More complex to centrally control, but more flexible
Data loss risk	Higher (if server fails without backup)	Very low (multiple full copies exist across machines)